SEMANA 01 - SESION 1

Modelos, tiempo, consistencia y trade-offs

Distributed systems - lo que cambia al cruzar la red

Sistemas Distribuidos (cod. 82739) - 2026-B, Grupo 2 - CORHUILA

Idea central de la sesion

"La correccion distribuida consiste en disenar primero los caminos de fallo. El camino feliz es la parte facil."

Que se trabajo

Al cruzar una frontera de proceso, un sistema pierde tres cosas que el monolito regalaba gratis: **memoria/estado compartido**, **reloj unico** y **fallo todo-o-nada**. Toda la sesion es la consecuencia de disenar contra esas tres perdidas.

Bloque	Que fija
8 falacias	La red no es confiable, la latencia no es cero, el ancho de banda no es infinito, la red no es segura, la topologia cambia, no hay un solo admin, el transporte cuesta, la red no es homogenea. Se disena con el **inverso** de cada una
Modelo de sistema/fallos	Asincrono + escalera crash-stop -> crash-recovery -> omision -> bizantino. Posicion elegida: **crash-recovery + omision**
Tiempo logico	Happens-before de Lamport ordena eventos, no el reloj de pared. L(a)<L(b) **no prueba** a->b; para eso hacen falta relojes vectoriales
Espectro de consistencia	linealizable -> secuencial -> causal -> eventual. **CAP/PACELC se elige por operacion**, nunca por sistema entero
Quorums	R + W > N garantiza que una lectura se solape con la ultima escritura (N=3,W=2,R=2 -> 4>3)
Consenso y entrega	FLP: consenso determinista es imposible en async puro con fallos; en la practica se usan timeouts/aleatoriedad. Raft = lider + log replicado, confirma por mayoria

El resultado mas importante: exactly-once delivery es un mito

Sobre una red no existe entrega exactamente-una-vez. Lo que si existe es **procesamiento exactamente-una-vez** = at-least-once + idempotency key + deduplicacion.

```
if (procesados.putIfAbsent(evento.id(), TRUE) == null) {
  aplicarEfecto(evento); }  // duplicados se ignoran, no se re-aplican
```

Ejemplo aplicado: "Comprar" durante una particion de red

Cliente pulsa Comprar. Orders reserva stock bien, y **justo despues se parte la red entre Orders y Payments**. El diseno ingenuo -una transaccion sincrona con lock de BD esperando a Payments- congela hilos, agota el pool y tumba la tienda entera. El diseno correcto:

#	Decision	Por que
1	No bloquear sosteniendo un lock	Convierte una falla local en caida total
2	Circuit breaker se abre y falla rapido	Deja de martillar a Payments particionado
3	Saga compensa: libera el stock	No hay ACID entre 3 servicios
4	Reintento del cobro con idempotency key	Al sanar la particion, no cobra 2 veces
5	Estado honesto: "confirmando pago..."	Un spinner infinito es una mentira

Y aqui es donde CAP/PACELC se aplica por operacion, no por sistema:

Modelos, tiempo, consistencia y trade-offs

Distributed systems - lo que cambia al cruzar la red

Operacion	Consistencia	Entrega
Reservar stock	Fuerte (linealizable)	at-least-once + idempotencia -> **C sobre A**
Cobrar pago	Fuerte (linealizable)	at-least-once + idempotency key
Ver mi pedido	Causal (read-your-writes)	-
Contador "vendidos"	Eventual	at-least-once -> **A sobre C**

Autoevaluacion de la sesion

7/7 completado - modelo de fallos, tiempo logico, consistencia por operacion, quorums, consenso y semanticas de entrega.

Distributed Systems - Semana 1, Sesion 1 - Faculty of Engineering, CORHUILA 2026-B. Material: code-corhuila.github.io/ova-web/2026-B/distribuidos/